GitHub Copilot dans VS Code

Bonnes pratiques avec les agents
Fil conducteur : HydroDataLab

20 octobre 2026 • Sources vérifiées les 18–19 octobre • Démonstration pédagogique, données publiques

Votre terrain de développement

Analystes BI et développeurs citoyens : Python, ETL et visualisation de données.

Développer

Explorer une idée.
Créer un traitement ou un visuel.

→

Tester sur le poste

Vérifier les calculs.
Rejouer avec des données d'essai.

→

Préparer le déploiement

Emballer code et dépendances pour un environnement conteneurisé.

Une marge pour expérimenter

Un cadre plus souple : l'occasion d'essayer, de comparer et d'apprendre.

La sécurité au quotidien

Données autorisées, secrets protégés.
Accès limités, code relu et testé.

> L'IA accélère les essais; vous gardez la maîtrise des données et des actions.

Microsoft · sécurité du développement avec les agents

Bien déléguer avec Copilot

Une mission précise, un contexte utile,
un résultat que vous pouvez vérifier.

Ce que vous saurez faire

Cadrer une session dans VS Code.
Choisir les règles et outils utiles.
Relire les changements et les tests.

Notre fil conducteur

Données publiques Hydro-Québec.
Une petite modification Python.
Un indicateur de qualité vérifiable.

> 45 minutes de présentation et de démos, puis vos questions.

Quatre séries, quatre sens différents

Série	Unité	Ce que l’on observe
Demande électrique au Québec	MW	Une puissance demandée
Niveau d’eau au réservoir Gouin	m	Une hauteur dans le référentiel source
Température minimale	°C	Un minimum sur la période source
Température maximale	°C	Un maximum sur la période source

Notre ETL : extraire les données, les transformer, puis charger ou exporter le résultat.
Un niveau en mètres ne donne pas une réserve en GWh.

Sources publiques HQ et contrat [S11, S12]. Instantané : 17 octobre 2026, 168 observations; récupéré le 18 octobre à 19:44 UTC.

Un modèle, un harnais, un agent

Notion	Dans notre tâche Python
Modèle	Propose du code et raisonne sur le résultat.
Outil · tool	Lit un fichier, le modifie ou lance Python.
Harnais · harness	Relie modèle, contexte, outils et autorisations.
Agent	Boucle sur ces actions pour accomplir votre tâche.

> **Notre choix : harnais Copilot dans VS Code.**
> Il utilise le Copilot SDK. Choisir un modèle Claude conserve ce harnais.

Microsoft · comprendre les harnais · Nouveautés 1.142

Git : garder une trace de ses essais

Mot rencontré	Ce qu'il signifie
Dépôt Git	Le projet et l'historique de ses versions enregistrées.
Commit	Un point enregistré dans cet historique.
Branche	Une suite de versions pour un essai. Exemple : `codex/repetition-01`.
Worktree · facultatif	Un autre dossier du même dépôt, pour travailler sur une autre branche.

> Pour débuter, une branche d'essai suffit.

Documentation Git · worktree [S22]. Les fichiers et résultats non enregistrés demandent leur propre sauvegarde.

Démo 1 · donner une mission à l'agent

> « Repère le calcul des indicateurs et explique comment il est testé.
> Ne modifie aucun fichier. »

Repère dans la fenêtre Agents	À quoi il sert
Session	Garder une conversation consacrée à cette tâche.
Planification	Demander une démarche avec `/plan` avant de modifier.
Autorisations	Choisir quelles actions doivent être confirmées.

Quelles sources permettent de vérifier sa réponse?

Microsoft · fenêtre Agents · Planification · Réglages

Écrire une règle que l'on peut vérifier

Exemple fictif : trois créneaux horaires de température attendus.

08 h	09 h	10 h
10 °C	**0 °C**	**None**
Mesure reçue	Mesure reçue	Aucune mesure

> **2 mesures valides sur 3 : couverture de 66,67 %.**
> Remplacer l'absence par 0 inventerait une troisième mesure.

Instruction permanente : « Conserver les valeurs manquantes. Compter 0 comme une mesure numérique valide. »

`.github/instructions/python.instructions.md`

Microsoft · instructions personnalisées · Se repérer dans Customizations →

Où écrire la règle, et quand s'applique-t-elle?

Règles communes du projet : `.github/copilot-instructions.md` · Markdown commun au projet.
Règle ciblée : `.github/instructions/python.instructions.md`

```
---
description: Qualité des transformations Python
applyTo: "etl/**/*.py"
---
Conserver les valeurs manquantes.
Compter 0 comme une mesure numérique valide.
```

applyTo : motif des fichiers créés ou modifiés. **description** : pertinence de la tâche.
Sans ces deux champs, joindre le fichier manuellement.

> Pour ce groupe : règles communes + règles Python ciblées.
> Plusieurs harnais compatibles? Préférer `AGENTS.md` pour les règles partagées.

Microsoft · format et activation · AGENTS.md

Tiered-context architecture

Architecture du contexte par paliers : **quoi charger, et quand?**

01 · Always on

Règles communes

- Dépôt + utilisateur
- Organisation, si activée
- Architecture : résumé et standards essentiels

Incluses si reconnues par le harnais.

02 · Task scoped

Selon la tâche

- Règles de chemin : `applyTo`
- Rôle et prompt choisis
- Skill chargée

Chargement conditionnel : fichier, rôle ou travail demandé.

03 · On demand

Au besoin

- Documents et code utiles
- Sélection IDE, contexte CLI
- Lectures et sortie terminal

Joint, récupéré par un outil ou fourni par l'interface.

Harnais : outils disponibles · serveurs MCP · permissions
Ils encadrent les actions. Configurer un serveur ne charge pas ses données.

Microsoft · conditions des instructions · Composition et contexte implicite

Comment ces apports composent le contexte

Exemple : « Ajoute le calcul de couverture et vérifie-le. »

1. **Always on** Règles communes
   copilot-instructions.md
2. **Task scoped** Règles Python
   applyTo: "etl/**/*.py"
3. **Task scoped** Rôle et demande
   « Ajoute la couverture, puis teste. »
4. **On demand** Documents et code
   DATA_DICTIONARY.md + coverage.py
5. **On demand** Résultats des outils
   pytest → 23/24 = 95,83 %

Model context

Contexte envoyé au modèle

Apport 1 · Commun

Préserver les données brutes. Garder les calculs testables hors réseau.

Les règles essentielles figurent dans les instructions du dépôt; le reste de la documentation se lit au besoin.

Aussi : système + historique. Vérifier **References**, lectures et résultats. Contexte limité.

Microsoft · assemblage du contexte · Ajouter les références utiles

Les personnalisations, dans VS Code

Overview : repères. **Customize Your Agent** : préparer des fichiers à relire.

Guider le travail

Instructions : règles du projet.
Skills : procédures réutilisables.
Agents : rôles spécialisés.

Donner des moyens d'agir

Tools : outils disponibles.
MCP Servers : connexions externes.

Automatiser et partager

Hooks : actions sur événement.
Plugins : paquets à partager.

Repères MCP pour VS Code 1.142

Transition récente : Copilot devient progressivement le harnais par défaut.

Copilot format `mcpServers`

Personnel, par défaut :
`%USERPROFILE%\.copilot\mcp-config.json`

Projet : `.mcp.json` à la racine*

Local format `servers`

Personnel : `mcp.json` du profil VS Code

Projet : `.vscode/mcp.json`

Emplacements conservés pour compatibilité.

* Destination encore expérimentale dans l'assistant d'ajout. Migration détaillée dans l'aide-mémoire.

Microsoft · version 1.142 · Fichiers MCP · Migration · Capture du panneau →

Skills et tools : deux choix distincts

Skill · charger une méthode

Choix automatique possible, ou `/nom` *

1 · Repérer
Nom + description
de la skill

→

2 · Choisir et charger
Corps du `SKILL.md`
dans le contexte

→

3 · Consulter au besoin
Fichiers de référence
et scripts associés

La méthode peut guider les outils. Un outil peut aussi servir sans skill.

Tool · exécuter une action

Selon la demande, le contexte et les outils disponibles

1 · Le modèle choisit
Outil + paramètres
ex. terminal → pytest

→

2 · Le harnais contrôle
Permissions, puis
exécution autorisée

→

3 · Le résultat revient
Le modèle décide
de la suite

> *** Notre skill `/validate-etl` est volontaire :** `disable-model-invocation: true`.
> Charger ses consignes ne lance pas ses scripts; leur exécution passe par des outils.

Microsoft · chargement progressif · Activation · Choix des outils

Une base « vanille », des ajouts ciblés

Mon point de départ : garder une configuration simple à faire évoluer.

Fonctions natives

Copilot, terminal, Git et tests. Des consignes de projet courtes.

→

Besoin récurrent

Ajouter un skill ou un outil pour une tâche précise.

→

Ajout entretenu

Relire, versionner et tester. Retirer ce qui ne sert plus.

Pourquoi rester simple?

Moins d’intégrations à adapter et de règles qui se contredisent.

Quand personnaliser?

Quand le gain est vérifiable : moins d’erreurs, de répétitions ou de reprises.

> Un skill ciblé peut aider. Accumuler les plugins n’est pas une stratégie de qualité.

Microsoft · migrations · Skills et chargement progressif

Les tokens : lire et produire

Un token est un morceau de texte :
un mot, une partie de mot ou un signe.

Ce que le modèle reçoit

Votre demande et les consignes.
Les extraits de code ou de données.
Les résultats des outils.

Ce que le modèle produit

Sa réponse et le code proposé.
Du raisonnement facturé
selon le modèle utilisé.

> Pour notre ETL : le schéma, un petit échantillon représentatif
> et les tests donnent un contexte utile.

Microsoft · contexte · GitHub · facturation. Évaluer toutes les tentatives nécessaires.

Auto : un bon point de départ

Niveau	Priorité	Exemple BI
Efficiency	Coût	Expliquer une fonction courte
Balance	Coût, qualité et temps d’attente	Transformer une série avec tests
Intelligence	Qualité	Investiguer une anomalie difficile

> Auto choisit selon la tâche et les modèles autorisés.

Plans payants : remise Auto de 10 % sur le coût des modèles.
Le volume total dépend du travail effectué.

GitHub · Auto model selection, consulté le 18 octobre 2026. Politiques et abonnement restent déterminants.

Comparer 4 tâches et 5 modèles

Auto Balance au quotidien · choix manuels ci-dessous

− 1 essai(s) +

Modèle Tokens entrée / sortie →	**Simple** Expliquer du code 3 000 / 500	**Délimitée** Modifier un ETL 20 000 / 2 000	**Complexe** Plusieurs fichiers 60 000 / 8 000	**Diagnostic difficile** Trouver une cause 60 000 / 8 000
GPT-6 Luna	◆ 0,00055	0,003	0,010	0,010
Claude Haiku 5.5	◆ 0,00055	0,003	0,010	0,010
Claude Sonnet 5.5	0,011	◆ 0,060	0,200	0,200
GPT-6.1 Sol	0,011	0,060	◆ 0,200	0,200
Claude Opus 5.5	0,022	0,120	0,400	◆ 0,400

◆ Choix conseillé à évaluer

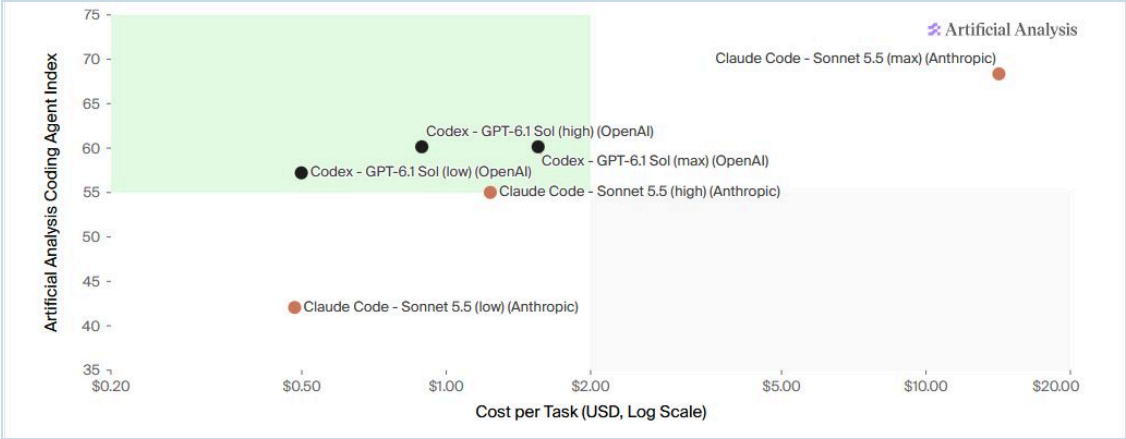
Échelle commune des barres : 0 → 0,400 $ US

Coûts simulés en $ US. Volumes fictifs par essai, identiques entre modèles pour chaque tâche.
Sans cache ni remise Auto. Comparer le coût total d'un résultat validé; ce n'est pas un benchmark.

GitHub · tarifs au 18 octobre 2026 · Comparer les modèles

Thinking effort et coût par tâche

Effort de raisonnement : partir du réglage recommandé, puis mesurer le gain.



19 octobre 2026 · indice v1.5 · coût : axe **logarithmique**.

Effort	Indice	$ US / tâche
GPT-6.1 Sol · Codex		
Low	57	0,50
High	60	0,89
Max	60	1,55
Sonnet 5.5 · Claude Code		
Low	42	0,48
High	55	1,24
Max	68	14,19

> **Sol High → Max :** coût +74 %; même indice affiché de 60.

Configurations Codex / Claude Code mesurées; coût API ≠ facture Copilot.

Microsoft · régler l'effort · Artificial Analysis · les six configurations ↗
Méthodologie · Coûts et indices affichés relevés le 19 octobre 2026.

Auto choisit; HydraFusion orchestre

Auto

Un modèle choisi pour la requête.
Préférence Efficiency, Balance ou Intelligence.

HydraFusion · préversion

Single : un modèle.
Cascade : proposition → jugement.
Critique : proposition → revue → révision.

> **Coût HydraFusion = somme des modèles utilisés.**
> Pas de supplément propre à HydraFusion; pas de remise Auto.
> Max augmente l'effort sur les tâches les plus exigeantes.

GitHub · fonctionnement et facturation HydraFusion
Annonce du 14 octobre · Documentation Auto

Démo 2 · vérifier un résultat

Données HQ → **Calcul Python** → **Tableau de bord**

Élément préparé	Son rôle
API .NET	Transmettre les commandes et les résultats à l'interface Vue.
Aspire	Démarrer les composants et regrouper leurs journaux.
Image Docker	Emballer le code et ses dépendances pour l'exécuter en conteneur.

> Quelles preuves faut-il avant d'accepter le résultat de l'agent?

Architecture HydroDataLab [S12]. Le calcul métier reste dans le même code Python.

SDD : une spécification avant le code

Spec-Driven Development · développement piloté par spécifications.
Le besoin devient des critères, puis un plan, du code et des preuves.
Notre règle : la **couverture** mesure la part des créneaux attendus avec une mesure valide.

Exemple sur une série horaire	Critère de réussite
24 mesures valides sur 24 attendues	100 %
23 mesures valides sur 24 attendues	95,83 %
Valeur zéro, doublon ou absence	Zéro compte; doublon et absence n'ajoutent rien.
Changement d'heure	23 ou 25 heures attendues selon la journée.

> Ces exemples deviennent des tests. Le SDD sera approfondi dans la prochaine formation animée par l'autre équipe.

GitHub · Spec Kit et développement piloté par spécifications

Démo 3 · détecter une mesure absente

24 mesures horaires attendues.
23 mesures numériques valides.

23 ÷ 24 = 95,83 %

> L’agent ajoute le calcul en Python.
> Nous vérifions les tests et l’indicateur affiché.

Scénario pédagogique : une mesure est retirée en mémoire. Les données brutes restent intactes.

Des skills à choisir selon votre tâche

Des pistes à évaluer dans Copilot, selon la tâche à accomplir.

Besoin	Point de départ	Utilité concrète
ETL Python	`validate-etl` · ce dépôt	Tests, unités et exports
Interface Vue	Playwright CLI · Microsoft	Parcours navigateur et captures
Application Aspire	Skills Aspire · officiels	Démarrage, santé et journaux
API .NET	dotnet/skills · équipe .NET	ASP.NET Core et tests ciblés
Cadrage et tests	Matt Pocock · grill-me, tdd	Clarifier le besoin; tester par étapes

> **Matt Pocock : en exploration.** Pas encore validé dans notre démo.

Microsoft · utiliser des skills partagés · Sélection au 19 octobre 2026

Votre prochaine tâche avec Copilot

Écrivez un résultat attendu, confiez une petite tâche, puis examinez les preuves.

Avant de déléguer

Définir un résultat attendu.
Joindre les fichiers utiles.
Vérifier règles et autorisations.

Avant d'accepter

Examiner le diff.
Lire les résultats des tests.
Conserver les décisions utiles.

> Quelle petite tâche pourriez-vous confier à Copilot dès demain?

Pour continuer et rester à jour

Suivre les évolutions

VS Code · nouveautés
Fonctions et migrations

Copilot · changelog
Modèles et services

Aspire · documentation
Orchestration et agents

Spec Kit · SDD
Spécifications et SDD

Apprendre et échanger

Microsoft Learn
Exercices guidés

Awesome Copilot
Exemples à relire et tester

GitHub Community
Échanges et retours d'expérience

Chez Hydro-Québec

Communauté BI / Python

Teams et échanges BI

Lien interne à ajouter

Guides et sécurité IA

Pratiques et outils autorisés

Lien interne à ajouter

Formations et contacts

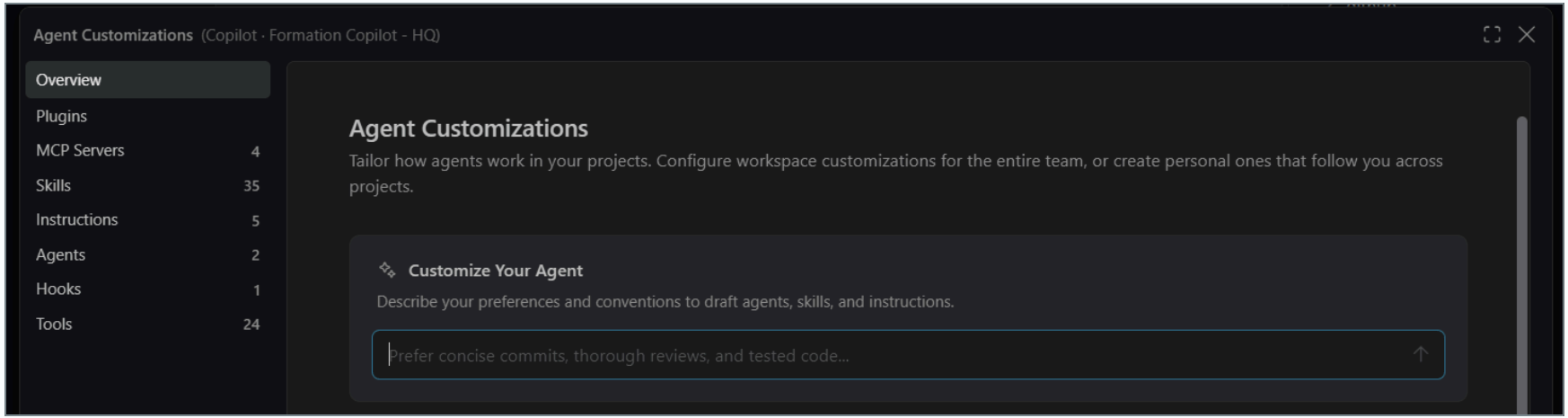
Parcours et contacts

Lien interne à ajouter

Routine proposée : 15 minutes de veille par semaine, puis une pratique testée sur votre projet.

Liens publics vérifiés le 19 octobre 2026. Ressources HQ à compléter. Canvas en préversion · annexe 28

Annexe · se repérer dans Customizations



Instructions → User

`~/.copilot/copilot-instructions.md`

Vos préférences communes aux projets.

Instructions → Workspace

`.github/copilot-instructions.md`

Les règles du projet, partagées dans Git.

Ouvrir une rubrique → examiner le fichier → vérifier son application.

Microsoft · panneau Customizations · Exemple de règle · Les rubriques →

Capture fournie le 19 octobre : vue Overview. Les chemins ci-dessus expliquent les portées dans Instructions; ils ne sont pas visibles dans la capture.

Annexe · modèles et tarifs Copilot

Modèle	Entrée	Cache lu	Cache écrit	Sortie
GPT-6 Luna	0,10	0,01	0,125	0,50
Claude Haiku 5.5	0,10	0,01	0,125	0,50
GPT-6.1 Sol	2,00	0,10	2,50	10,00
Claude Sonnet 5.5	2,00	0,10	2,50	10,00
Claude Opus 5.5	4,00	0,20	5,00	20,00

$ US / million de tokens, tarifs standards. 1 AI credit = 0,01 $ US.
Long contexte : Haiku > 100k; Luna et Sol > 272k → grille distincte.

GitHub · grille officielle des tarifs, 18 octobre 2026. Disponibilité HQ à confirmer.

Annexe · Canvas, travailler dans le même objet



Schéma pédagogique : une extension fournit l'interface et ses actions.
Ce dessin et le tableau de bord Vue ne sont pas des Canvas VS Code.

Ouvrir une surface

Activer `chat.canvases.enabled`.
Créer une nouvelle session Copilot.
`/create-canvas` si disponible;
puis **+** → **Canvas** pour un canvas installé.

Cadre actuel · Preview

Application VS Code de bureau.
Session locale Copilot / Agent Host.
Pas le harnais « Local », ni Claude,
Codex ou une session distante.

Microsoft · prérequis et fonctionnement · Nouveauté 1.142 · Retour →

Annexe · le contrat d'une observation

Dimension	Question à résoudre
Schéma et identité	Quelle série, station et instant unique?
Unité et sens	MW, m ou °C? Mesure, minimum, maximum?
Fréquence	15 minutes, horaire ou quotidien?
Temps	Instant UTC; journée America/Toronto?
Provenance	URL, licence, récupération et version brute?
Qualité	Null, doublon, invalidité et couverture attendue?

Contrat HydroDataLab [S12] et métadonnées publiques HQ [S11]. Préserver le sens de la source.

Annexe · reprendre sans improviser

00 socle → **01 instructions** → **02 extraction** → **03 BI** → **04 pilotage** → **05 Docker** → **06 spec** → **07 solution**

- Un tag Git est un repère nommé sur une version enregistrée.
- Conserver ses modifications avant de changer de version.
- Séparer les données d’exécution des fichiers de code.
- Reprendre avec la copie locale des données et les tests.

> PDF, solution préparée et Git pour reprendre.

Guide présentateur et procédure de reprise du projet. Les checkpoints VS Code n’annulent pas les effets externes des outils [S06].

Annexe · sources et limites

VS Code et GitHub

Personnalisations
Contexte
Auto
Tarifs Copilot
HydraFusion, 14 octobre

Données et pratique

Données ouvertes HQ
Microsoft Learn
Awesome Copilot
Spec Kit

> Sources consultées les 18 et 19 octobre 2026. Les fonctions disponibles dépendent de la version et du compte.

Registre complet : docs/SOURCES.md. Schémas pédagogiques; aucune capture VS Code inventée.